

MATH 231 – Linear Algebra I

The Computational Unit, Part 6 ■ Gradient Descent

Kiely Hall 326 ■ Instructor: Kevin Bedoya

About this document. The vector calculus unit established that the gradient points in the direction of steepest ascent, and remarked that an algorithm follows from it. This is that algorithm. The whole of it is: to find a low point, keep walking downhill.

Part	Sections	Subject
1	§1–§4	operation counting, Big-O, floating point, error
2	§5–§10	k -nearest neighbours
3	§11–§15	k -means clustering
4	§16–§20	the Gram–Schmidt process
5	§21–§23	matrix multiplication as an algorithm
6	§24–§27	gradient descent

Assumed. From MATH 141: the derivative, and finding minima by setting it to zero. From the vector calculus unit: the gradient (§7.1) and, above all, that ∇f points in the direction of steepest ascent while $-\nabla f$ points in the direction of steepest descent (§7.3). From this unit: tolerances and floating-point comparison (§4), and local minima as a failure mode (§14.1).

What you should be able to do afterwards. Say when the analytic method for finding a minimum stops being available; state the descent step and explain why it moves against the derivative; explain what the learning rate does and what goes wrong when it is too large or too small; write the pseudocode with its stopping test and iteration cap; say what the algorithm does and does not promise; and write the same algorithm in n dimensions using the gradient.

24. The problem, and the obvious method

24.1. A function to minimise

Start with the simplest interesting question in the subject: given a function f , find the input at which it is smallest.

Take

$$f(x) = x^2.$$

The answer is visible without any machinery. The graph is a parabola opening upward, it is never negative, and it equals zero at $x = 0$. So the minimum is at $x^* = 0$ with value $f(x^*) = 0$.

24.2. The analytic method

The method from MATH 141 confirms it. At a minimum the tangent line is flat, so the derivative is zero there. Differentiate, set to zero, solve:

$$f'(x) = 2x, \quad 2x = 0, \quad x^* = 0.$$

Three steps, an exact answer, no approximation anywhere. When this works it is unbeatable.

24.3. Where it stops working

Notice what the analytic method actually required: that the equation $f'(x) = 0$ could be *solved in closed form*. For $2x = 0$ that is trivial. It is not generally so.

Consider

$$f(x) = x^6 - 3x^4 + e^x + \frac{1}{1+x^2}.$$

Differentiating is mechanical — every rule needed is in the tables. But the resulting equation $f'(x) = 0$ mixes a polynomial with an exponential and a rational function, and there is no algebraic procedure that isolates x . The derivative exists, we can evaluate it at any point we like, and we still cannot solve for the point where it vanishes.

This is the ordinary situation, not a contrived one. In the applications this course is heading toward, f may depend on thousands of variables and be assembled from data rather than written as a formula. Setting a gradient to zero and solving is out of the question.

So we need a method that never solves anything — one that only ever *evaluates* f and f' at points of our choosing, and uses those numbers to move somewhere better.

25. Descending by steps

25.1. The idea

Here is the whole plan. Start at a guess. Look at which way is downhill. Take a step that way. Repeat until the ground is flat.

From the vector calculus unit. §7.3 supplies the one fact this needs. At any point, f increases fastest in the direction of the gradient — in one variable, the direction in which the derivative points. To *decrease* f as fast as possible, move in the opposite direction.

In one variable that reads: if $f'(x_k) > 0$ then f is rising to the right, so step left; if $f'(x_k) < 0$ then f is rising to the left, so step right. Both cases are captured by subtracting the derivative, since subtracting a positive number moves left and subtracting a negative number moves right.

25.2. A first attempt

The simplest rule that does this is

$$x_{k+1} = x_k - f'(x_k). \tag{1}$$

Start somewhere, subtract the derivative, repeat, and stop when $|f'(x_k)|$ is small enough that the ground is essentially flat.

Test it on $f(x) = x^2$, where $f'(x) = 2x$. Starting from $x_0 = 1$:

$$x_{k+1} = x_k - 2x_k = -x_k.$$

So the sequence is

$$1, -1, 1, -1, 1, \dots$$

It jumps across the minimum to the mirror-image point, jumps back, and repeats forever. It never gets closer to $x^* = 0$, and since $|f'(x_k)| = 2$ at every step, the stopping test never fires either. The loop does not terminate.

And this is the easiest possible function.

The direction was never the problem — at $x = 1$ the derivative is $+2$, and moving left is correct. The problem is the *distance*. The rule moved a step of size 2 when the minimum was only 1 away, so it overshoot, and it overshoots by the same amount forever.

25.3. The learning rate

The fix is to stop taking the derivative at face value as a distance, and take only a fraction of it. Introduce a positive number η that scales the step:

$$\boxed{x_{k+1} = x_k - \eta f'(x_k)} \quad (2)$$

Definition 25.1 (Learning rate). The number $\eta > 0$ in (2) is the **learning rate**. It is a **hyperparameter**: a value chosen by whoever runs the algorithm, not computed by it, and not determined by f .

The derivative still chooses the direction. The learning rate decides how far to commit to it. Small η means cautious steps; large η means aggressive ones.

25.4. Choosing it

On $f(x) = x^2$ we can see the entire effect of η exactly, because the iteration solves in closed form. With $f'(x) = 2x$,

$$x_{k+1} = x_k - \eta(2x_k) = (1 - 2\eta)x_k \quad \implies \quad x_k = (1 - 2\eta)^k x_0.$$

Every step multiplies the distance to the minimum by the fixed factor $1 - 2\eta$. So the behaviour depends entirely on that one number:

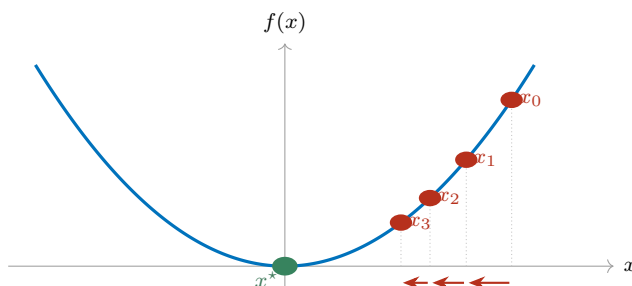
Learning rate	Factor $1 - 2\eta$	What happens
$\eta = 0.1$	0.8	shrinks steadily; converges
$\eta = 0.5$	0	lands exactly on x^* in one step
$\eta = 0.9$	-0.8	overshoots each time, but by less; still converges
$\eta = 1$	-1	overshoots exactly; oscillates forever
$\eta = 1.1$	-1.2	overshoots by more each time; diverges

Convergence requires $|1 - 2\eta| < 1$, that is $0 < \eta < 1$. The failure of (1) was the row $\eta = 1$ — the naive rule was gradient descent with the learning rate accidentally set to the one value that oscillates forever.

Example 25.1 (A run with $\eta = 0.1$). Starting from $x_0 = 1$, each step multiplies by 0.8:

k	x_k	$f(x_k)$	$f'(x_k)$
0	1	1	2
1	0.8	0.64	1.6
2	0.64	0.4096	1.28
3	0.512	0.262144	1.024
4	0.4096	0.167772	0.8192
5	0.32768	0.107374	0.65536

The iterates never reach 0 exactly — they approach it geometrically, which is why a tolerance is needed rather than a test for equality. Reaching $|f'(x_k)| < 10^{-7}$ takes 76 steps.



Each step is smaller than the last, because the derivative — which sets the step size — shrinks as the curve flattens. The algorithm slows down automatically as it approaches the bottom.

A common confusion (Bigger is not faster). It is tempting to read “ η controls how fast we descend” as “larger η means faster”. The table says otherwise: past $\eta = 0.5$ the method starts overshooting, and past $\eta = 1$ it diverges outright, running away from the very minimum it was asked to find. Too small is merely slow; too large is fatal.

25.5. Why it is called a *learning rate*

The name comes from machine learning, and the idea underneath it is simple.

A model is a rule with adjustable numbers inside it, called **weights**. Feed it an input and it produces a guess. Compare that guess with the correct answer and you get an error. Do this across all the available data and add the errors up, and you have a single number measuring how wrong the model currently is. That number is called the **loss**.

The loss depends on the weights — change them and it changes. So the loss is a function, the weights are its inputs, and *making the model better means making that function small*. Which is the problem this document is about.

Gradient descent does the adjusting. At each step it computes which way to nudge the weights to reduce the error, and nudges them. Repeating that is what “learning” means here: a model learns by being shown that it was wrong and correcting itself slightly in response.

The learning rate is the size of that correction. Set it small and the model edges toward better weights slowly but steadily. Set it large and it lurches — possibly past the answer entirely, exactly as in the $\eta = 1.1$ row above. Hence the name: η controls how fast the model learns, and whether it learns at all.

26. The algorithm

26.1. Pseudocode

Gradient descent (one variable)

Input: f' , a starting point x_0 , a learning rate $\eta > 0$, a tolerance $\tau > 0$, an iteration cap $k_{\max}$.

Output: a point where $|f'| < \tau$, or a report of failure.

```
1:  $x \leftarrow x_0$ 
2: for  $k = 0, 1, 2, \dots, k_{\max}$ :
3:    $g \leftarrow f'(x)$                                 the slope where we are standing
4:   if  $|g| < \tau$ : return  $x$                             flat enough — stop
5:    $x \leftarrow x - \eta g$                                 step downhill
6: return failure —  $k_{\max}$  steps without reaching the tolerance
```

Three of these lines deserve comment.

Line 4 tests before stepping. If the starting point already satisfies the tolerance, the algorithm returns it immediately rather than taking a pointless step first.

The tolerance τ is a comparison against a small number, not against zero. As §25.4 showed, the iterates approach the minimum without ever landing on it, so a test for $f'(x) = 0$ would never fire. Typical choices are $\tau = 10^{-7}$ for single precision or $\tau = 10^{-14}$ for double, matching the precisions of §3.3 — there is no point demanding an accuracy the arithmetic cannot represent.

The cap $k_{\max}$ is not boilerplate. The $\eta = 1$ row of §25.4 runs forever without ever satisfying the test on line 4. Without line 2's bound the procedure would simply never return. A cap converts a hang into a reported failure, which is the difference between a program you can debug and one you have to kill.

26.2. What it does not promise

The algorithm returns a point where the derivative is small. It is worth being precise about what that does and does not mean.

A small derivative marks a *critical point* — somewhere the graph is locally flat. Minima are flat, but so are maxima and inflection points. In practice descent will not settle on a maximum, since any step away from one decreases f and the method never comes back; but a flat stretch or a local dip will hold it.

More importantly, “a minimum” is not “the minimum”. If f has several valleys, the algorithm finds whichever one the starting point drains into. A different x_0 can give a different answer, and nothing in the procedure can tell you whether a lower valley exists somewhere it never looked.

This is the same limitation §14.1 identified for k -means, and the same remedy applies: run it several times from different starting points and keep the best result. An algorithm that walks downhill can only report on the hill it was dropped onto.

27. In n dimensions

27.1. The generalisation

Nothing above was really about one variable. The argument was: move opposite to the direction of steepest increase, by an amount controlled by η . In $\mathbb{R}^n$ that direction has a name — it is $-\nabla f$ — and the rule transfers with no change of idea at all.

Definition 27.1 (Gradient descent). Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Given a starting point $\mathbf{x}_0 \in \mathbb{R}^n$ and a

learning rate $\eta > 0$, **gradient descent** generates the sequence

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta \nabla f(\mathbf{x}_k), \quad (3)$$

stopping when $\|\nabla f(\mathbf{x}_k)\| < \tau$.

Set the two versions side by side:

	One variable	n variables
The point	$x_k \in \mathbb{R}$	$\mathbf{x}_k \in \mathbb{R}^n$
The direction	$f'(x_k)$	$\nabla f(\mathbf{x}_k)$
The step	$x_k - \eta f'(x_k)$	$\mathbf{x}_k - \eta \nabla f(\mathbf{x}_k)$
The stopping test	$ f'(x_k) < \tau$	$\ \nabla f(\mathbf{x}_k)\ < \tau$

Three substitutions and nothing else: the number becomes a vector, the derivative becomes the gradient, and the absolute value becomes the norm. The learning rate stays a single scalar — one number scaling the whole vector step, not one per coordinate.

The pseudocode of §26.1 carries over under those substitutions; §27.4 writes it out in full.

27.2. An example

Example 27.1. Minimise $f(x, y) = x^2 + y^2$, whose minimum is plainly at the origin. From the vector calculus unit, $\nabla f(x, y) = [2x, 2y]$, so (3) reads

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \eta [2x_k, 2y_k] = (1 - 2\eta) \mathbf{x}_k.$$

This is the one-variable result again, now applied to both coordinates at once: the whole vector shrinks by the factor $1 - 2\eta$ every step. Starting from $\mathbf{x}_0 = [1, 2]$ with $\eta = 0.1$,

$$[1, 2] \rightarrow [0.8, 1.6] \rightarrow [0.64, 1.28] \rightarrow [0.512, 1.024] \rightarrow \dots$$

converging to $[0, 0]$ along the straight line through the origin.

27.3. One learning rate, many directions

The example above was unusually kind: the function curves at the same rate in both directions, so a single η suits both equally.

Change it to

$$f(x, y) = x^2 + 10y^2,$$

a bowl far steeper along y than along x , and the two coordinates now want different treatment. The gradient is $[2x, 20y]$, so a step multiplies the x -part by $1 - 2\eta$ and the y -part by $1 - 20\eta$. Convergence needs *both* factors below 1 in size, which forces $\eta < 0.1$ — a restriction imposed entirely by the steep direction. The shallow x direction is then crawling, multiplied by at least 0.8 each step, purely because a larger η would make y diverge.

The single scalar η must serve every direction at once, and the worst direction sets the limit.

Looking ahead. That tension is the starting point for most of what follows. Everything that improves on plain gradient descent is, in one way or another, an attempt to stop the steepest

direction from dictating the step size for all the others — by adapting η as the run proceeds, by giving each coordinate its own effective rate, or by using curvature information to rescale the step. The quantity that measures how badly a problem suffers from this also has a name, and it is one this unit has met before in a different guise.

27.4. The general algorithm

Collecting everything, here is the method in its general form. It is the procedure of §26.1 with the three substitutions of §27.1 applied, and it is what “gradient descent” refers to without further qualification.

Gradient descent

Input: the gradient ∇f of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$; a starting point $\mathbf{x}_0 \in \mathbb{R}^n$; a learning rate $\eta > 0$; a tolerance $\tau > 0$; an iteration cap $k_{\max}$. **Output:** a point where $\|\nabla f\| < \tau$, or a report of failure.

```

1:  $\mathbf{x} \leftarrow \mathbf{x}_0$  start from the initial guess
2: for  $k = 0, 1, 2, \dots, k_{\max}$ :
3:    $\mathbf{g} \leftarrow \nabla f(\mathbf{x})$  steepest ascent at the current point
4:   if  $\|\mathbf{g}\| < \tau$ : return  $\mathbf{x}$  flat enough — stop
5:    $\mathbf{x} \leftarrow \mathbf{x} - \eta \mathbf{g}$  step against the gradient
6: return failure —  $k_{\max}$  steps without reaching the tolerance

```

Every line is the one-variable line with a vector in place of a number. Line 3 evaluates the gradient rather than the derivative; line 4 measures it with the norm rather than the absolute value; line 5 subtracts a scaled vector rather than a scaled number. The learning rate η , the tolerance τ , and the cap $k_{\max}$ are unchanged — all three remain single scalars, whatever n is.

The three inputs the caller must supply are worth naming one final time, since none of them is computed by the algorithm and all three change what it does: η decides whether it converges at all (§25.4), τ decides when it is satisfied (§26.1), and $\mathbf{x}_0$ decides which minimum it finds (§26.2).